# Lecture 3: Binary Search

Binary search finds the position of a target value within a sorted array.

It compares the target with the middle element. If they are equal, the search ends.

If the target is smaller than the middle element, the search continues in the left half;

if it is larger, the search continues in the right half.

Each comparison halves the search range, so the time complexity is O(log n).

The array MUST be sorted. On an unsorted array binary search gives wrong results.

Example: A = [2, 5, 8, 12, 16, 23, 38], target = 16.

low = 0, high = 6, mid = 3, A[3] = 12 < 16, so low = 4.

mid = 5, A[5] = 23 > 16, so high = 4. mid = 4, A[4] = 16. Found at index 4.

Space complexity of iterative binary search is O(1); recursive version uses O(log n) stack.

# Lecture 3: Binary Search Trees

A binary search tree (BST) stores keys so that for every node,

all keys in the left subtree are smaller and all keys in the right subtree are larger.

Search, insert and delete take O(h) time where h is the height of the tree.

For a balanced tree h = O(log n); for a degenerate (skewed) tree h = O(n).

Inorder traversal of a BST visits keys in sorted order.

# Lecture 7: Deadlock

A deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource held by some other process.
Four necessary conditions (Coffman conditions): mutual exclusion, hold and wait, no preemption, and circular wait. All four must hold for a deadlock to occur.
Deadlock prevention ensures at least one of the four conditions can never hold.
Example: impose a total ordering on resource types and require each process to request resources in increasing order. This prevents circular wait.
Deadlock avoidance (e.g. Banker's algorithm) decides at run time whether granting a request keeps the system in a safe state. Prevention and avoidance are different strategies.

# Lecture 8: Graph Traversal · BFS and DFS

Breadth-first search (BFS) explores a graph level by level using a queue.

BFS finds the shortest path (fewest edges) in an unweighted graph.

Depth-first search (DFS) goes as deep as possible before backtracking, using a stack or recursion.

Both BFS and DFS run in O(V + E) time with an adjacency list.

DFS is used for topological sort, cycle detection and finding connected components.